

Banco de Questões Objetivas – Aula 11

Técnicas de Programação SQL

Banco de Dados

Instruções

Este banco contém questões objetivas sobre os principais tópicos da Aula 11:

- programação para banco de dados;
- linguagem hospedeira e sublinguagem de dados;
- descompasso de impedância;
- SQL embutido;
- cursores;
- SQL dinâmica;
- SQLJ;
- JDBC;
- Hibernate;
- SQL/CLI;
- procedimentos e funções armazenadas;
- SQL/PSM;
- SQL Injection;
- dados criptografados.

1 Questões de Múltipla Escolha

Q1. Em aplicações reais, a maioria das interações com bancos de dados ocorre por meio de:

- a) programas de aplicação cuidadosamente projetados e testados.
- b) digitação manual de comandos pelo administrador.
- c) arquivos de texto sem conexão com SGBD.
- d) consultas feitas apenas por planilhas.

Q2. A interface interativa de um SGBD é conveniente principalmente para:

- a) criação de esquema, criação de restrições e consultas ad-hoc.
- b) processamento automático de todas as aplicações web.
- c) substituição completa de programas de aplicação.
- d) eliminação de conexões com o servidor.

Q3. Em programação de banco de dados, a linguagem hospedeira é:

- a) a linguagem de programação geral usada pela aplicação, como Java, C ou C++.
- b) sempre a linguagem SQL.
- c) o nome físico da tabela no disco.
- d) uma restrição de integridade.

Q4. Em programação de banco de dados, a sublinguagem de dados geralmente é:

- a) SQL.
- b) HTML.
- c) CSS.
- d) Assembly.

Q5. Uma das abordagens para programação de BD é:

- a) comandos de BD embutidos em uma linguagem de programação.
- b) ausência total de linguagem hospedeira.
- c) uso exclusivo de arquivos PDF.
- d) proibição de bibliotecas de acesso.

Q6. A abordagem baseada em biblioteca de funções de BD consiste em:

- a) disponibilizar uma API para chamadas ao banco a partir da linguagem hospedeira.
- b) colocar SQL dentro de imagens.
- c) impedir conexão com vários bancos.
- d) substituir todas as tabelas por arquivos texto.

Q7. Uma linguagem completamente nova para BD é ideal principalmente quando:

- a) a aplicação possui interação intensiva com o banco de dados.
- b) não existe banco de dados.
- c) deseja-se evitar qualquer acesso ao SGBD.
- d) o banco deve ser usado apenas manualmente.

Q8. O descompasso de impedância ocorre por causa da:

- a) incompatibilidade entre o modelo da linguagem hospedeira e o modelo do banco.
- b) igualdade perfeita entre Java e SQL.
- c) inexistência de tipos de dados no banco.
- d) impossibilidade de usar cursores.

Q9. Um exemplo de descompasso de impedância é:

- a) diferenças entre tipos de dados da linguagem hospedeira e do BD.
- b) uma tabela possuir chave primária.
- c) uma consulta ter **WHERE**.
- d) o banco possuir apenas uma tabela.

Q10. Resultados de consultas SQL geralmente são:

- a) bags de tuplas, isto é, sequências de valores de atributos.
- b) sempre um único número inteiro.
- c) arquivos executáveis.
- d) classes Java obrigatoriamente.

Q11. Para processar múltiplas tuplas retornadas por uma consulta em uma linguagem hospedeira, normalmente usa-se:

- a) cursor ou iterador.
- b) comando **DROP**.
- c) chave estrangeira.
- d) criptografia MD5 obrigatória.

Q12. A interação básica de um programa cliente com o BD segue a ordem:

- a) abrir conexão, submeter comandos, fechar conexão.
- b) fechar conexão, submeter comandos, abrir conexão.
- c) criar índice, apagar tabela, abrir conexão.
- d) executar **DROP**, executar **DELETE**, encerrar o SGBD.

Q13. SQL embutido significa que:

- a) comandos SQL são inseridos dentro de uma linguagem de programação geral.
- b) comandos SQL são proibidos dentro de programas.
- c) o banco é acessado apenas por interface gráfica.
- d) o SGBD deixa de usar SQL.

Q14. Em SQL embutido, os comandos SQL podem incluir:

- a) definições de dados, restrições, consultas, atualizações e visões.
- b) apenas comandos **SELECT**.

- c) apenas comandos DROP.
- d) somente comentários.

Q15. Em SQL embutido, comandos como EXEC SQL servem para:

- a) distinguir comandos SQL dos comandos da linguagem hospedeira.
- b) apagar o banco automaticamente.
- c) declarar uma chave primária em Java.
- d) criar uma conexão HTTP.

Q16. Variáveis compartilhadas em SQL embutido geralmente aparecem prefixadas por:

- a) dois pontos, como :variavel.
- b) arroba, como @variavel.
- c) sustenido, como #variavel.
- d) cifrão, como \$variavel.

Q17. O pré-processador em SQL embutido pode:

- a) separar comandos SQL embutidos do programa na linguagem hospedeira.
- b) remover todas as tabelas do banco.
- c) substituir o SGBD por um arquivo local.
- d) impedir a compilação da aplicação.

Q18. No SQL embutido em C, variáveis declaradas em uma seção DECLARE são usadas para:

- a) compartilhar valores entre o programa e comandos SQL.
- b) criar automaticamente todas as tabelas.
- c) impedir comunicação com o banco.
- d) apagar exceções do compilador.

Q19. Em SQL embutido, variáveis como SQLCODE ou SQLSTATE são usadas para:

- a) comunicar erros ou exceções entre o BD e o programa.
- b) armazenar a senha criptografada do usuário.
- c) substituir todos os cursores.
- d) criar uma nova linguagem de programação.

Q20. O comando CONNECT TO server-name AS connection-name AUTHORIZATION user-account-info é usado para:

- a) conectar o programa a um servidor de banco de dados.
- b) fechar todas as conexões.
- c) criar uma tabela.
- d) executar uma consulta aninhada.

Q21. O comando `SET CONNECTION connection-name;` é usado para:

- a) mudar a conexão ativa.
- b) remover uma conexão do banco.
- c) apagar a conexão fisicamente.
- d) criar uma função armazenada.

Q22. O comando `DISCONNECT connection-name;` é usado para:

- a) desconectar uma conexão.
- b) criar uma conexão.
- c) mudar a conexão ativa.
- d) criar uma trigger.

Q23. Em SQL embutido, quando uma consulta retorna múltiplas tuplas, é necessário usar:

- a) cursor.
- b) `DROP TABLE`.
- c) chave primária composta.
- d) `DELETE` sem `WHERE`.

Q24. O comando `FETCH` em um cursor serve para:

- a) mover o cursor para a próxima tupla.
- b) apagar todas as tuplas.
- c) criar uma nova conexão.
- d) declarar uma procedure.

Q25. O comando `CLOSE CURSOR` indica que:

- a) o processamento da consulta foi completado.
- b) uma tabela foi removida.
- c) uma conexão foi aberta.
- d) uma senha foi criptografada.

Q26. A SQL dinâmica permite:

- a) compor e executar declarações SQL em tempo de execução.
- b) executar apenas consultas previamente compiladas.
- c) impedir qualquer comando digitado pelo usuário.
- d) substituir SQL por XML.

Q27. Na SQL embutida tradicional, cada nova consulta pode exigir:

- a) alteração do código-fonte.

- b) remoção do banco.
- c) desligamento do servidor.
- d) criação de uma nova linguagem.

Q28. Uma atualização dinâmica costuma ser mais simples que uma consulta dinâmica porque:

- a) em consultas, tipos e números de atributos recuperados podem ser desconhecidos em tempo de compilação.
- b) atualizações nunca usam SQL.
- c) consultas dinâmicas não retornam dados.
- d) atualizações sempre removem tabelas.

Q29. SQLJ é:

- a) um padrão para embutir SQL em Java.
- b) uma linguagem para apagar bancos.
- c) uma forma de criptografia de senha.
- d) um tipo de chave estrangeira.

Q30. O tradutor SQLJ transforma comandos SQLJ em Java usando a interface:

- a) JDBC.
- b) HTML.
- c) CSS.
- d) XML puro.

Q31. Em SQLJ, para processar várias tuplas, são usados:

- a) iteradores.
- b) comandos DROP.
- c) chaves estrangeiras.
- d) triggers obrigatórias.

Q32. Em SQLJ, um iterador designado é associado ao resultado de uma consulta por meio de:

- a) lista de tipos e nomes dos atributos.
- b) apenas nomes de tabelas.
- c) apenas senhas de usuário.
- d) apenas comandos DELETE.

Q33. Em SQLJ, um iterador posicional lista:

- a) somente os tipos dos atributos.

- b) somente os nomes dos atributos.
- c) somente as tabelas do banco.
- d) somente os usuários conectados.

Q34. JDBC significa:

- a) Java Database Connectivity.
- b) Java Data Backup Control.
- c) Join Database Command.
- d) Java Domain Constraint.

Q35. JDBC é uma:

- a) biblioteca de funções de BD disponível para Java.
- b) linguagem completamente nova que substitui Java.
- c) técnica exclusiva para apagar bancos.
- d) criptografia de senha.

Q36. Um programa Java com JDBC pode acessar qualquer SGBD relacional que possua:

- a) driver JDBC.
- b) arquivo PDF.
- c) comando `DROP`.
- d) senha em texto puro.

Q37. JDBC permite que um programa:

- a) conecte-se a vários bancos ou fontes de dados.
- b) acesse apenas um banco fixo sem driver.
- c) elimine a necessidade de SQL.
- d) funcione sem conexão.

Q38. Um dos primeiros passos no acesso a BD com JDBC é:

- a) importar a biblioteca `java.sql.*`.
- b) executar `DROP TABLE`.
- c) fechar a conexão antes de abrir.
- d) criptografar todas as tabelas.

Q39. Em JDBC, um objeto `Connection` representa:

- a) uma conexão com o banco de dados.
- b) uma tupla retornada.
- c) uma coluna da tabela.

d) uma função armazenada.

Q40. Em JDBC, um objeto `Statement` representa:

- a) um comando SQL a ser executado.
- b) uma conexão física com a internet.
- c) uma tabela resultante.
- d) uma senha criptografada.

Q41. Em JDBC, parâmetros de comandos SQL podem ser indicados por:

- a) ponto de interrogação, ?.
- b) asterisco, *.
- c) dois pontos, :.
- d) barra, /.

Q42. Em JDBC, `PreparedStatement` é recomendado principalmente quando:

- a) a consulta será executada múltiplas vezes ou possui parâmetros.
- b) a consulta nunca será executada.
- c) o objetivo é apagar a conexão.
- d) o banco não suporta SQL.

Q43. Em JDBC, antes de executar um `PreparedStatement`, os parâmetros devem ser:

- a) ligados a variáveis do programa.
- b) apagados do banco.
- c) transformados em tabelas.
- d) convertidos obrigatoriamente para XML.

Q44. Em JDBC, `executeQuery` é usado principalmente para:

- a) comandos de consulta que retornam `ResultSet`.
- b) comandos `INSERT`, `DELETE` e `UPDATE`.
- c) criação de classes Java.
- d) fechamento de conexão.

Q45. Em JDBC, `executeUpdate` é usado principalmente para:

- a) comandos `INSERT`, `DELETE` ou `UPDATE`.
- b) consultas que retornam `ResultSet`.
- c) abrir conexão.
- d) criar driver.

Q46. Em JDBC, `ResultSet` pode ser entendido como:

- a) uma tabela bidimensional com o resultado da consulta.
- b) uma chave primária.
- c) uma conexão ativa.
- d) uma procedure armazenada.

Q47. Em JDBC, o método `next()` de um `ResultSet` serve para:

- a) avançar para a próxima linha do resultado.
- b) apagar a próxima linha da tabela.
- c) fechar a conexão.
- d) criar uma nova tabela.

Q48. Hibernate facilita principalmente:

- a) o mapeamento objeto-relacional.
- b) a remoção de todos os bancos relacionais.
- c) a substituição de classes Java por SQL puro.
- d) a criação de senhas MD5.

Q49. Hibernate pode transformar:

- a) classes Java em tabelas de dados e vice-versa.
- b) consultas SQL em imagens.
- c) conexões em arquivos PDF.
- d) senhas em chaves primárias.

Q50. Uma vantagem do Hibernate apresentada na aula é:

- a) gerar chamadas SQL e liberar o desenvolvedor de conversões manuais.
- b) impedir portabilidade entre SGBDs.
- c) eliminar completamente a necessidade de banco de dados.
- d) substituir o SGBD por HTML.

Q51. Em Hibernate, um arquivo XML pode ser usado para:

- a) relacionar propriedades do objeto aos campos da tabela.
- b) apagar usuários do banco.
- c) executar `DELETE` sem `WHERE`.
- d) substituir a linguagem Java.

Q52. DAO, no contexto do exemplo de Hibernate, aparece como uma classe responsável por:

- a) persistir o objeto.
- b) criptografar todas as senhas.

- c) apagar o servidor.
- d) substituir o driver JDBC.

Q53. SQL embutido provê principalmente:

- a) programação de BD estática.
- b) programação dinâmica via API.
- c) ausência de pré-processamento.
- d) execução apenas em tempo de execução.

Q54. APIs de acesso a BD oferecem como vantagem:

- a) não precisar de pré-processamento, sendo mais flexíveis.
- b) checagem completa do SQL em tempo de compilação.
- c) impossibilidade de conexão.
- d) ausência de funções.

Q55. Uma desvantagem de APIs de acesso a BD é que:

- a) a checagem do SQL é feita em tempo de execução.
- b) nunca permitem SQL dinâmico.
- c) exigem sempre pré-processador.
- d) não podem usar strings.

Q56. SQL/CLI significa:

- a) Call Level Interface.
- b) Command Line Internet.
- c) Create Logic Index.
- d) Control Language Instance.

Q57. SQL/CLI faz parte:

- a) do padrão SQL.
- b) apenas do HTML.
- c) do sistema operacional.
- d) de uma linguagem sem relação com BD.

Q58. Em SQL/CLI, comandos SQL são:

- a) criados dinamicamente e passados como parâmetros string.
- b) proibidos dentro de programas.
- c) sempre previamente compilados.
- d) substituídos por imagens.

Q59. Um dos componentes de SQL/CLI é:

- a) registro de ambiente.
- b) registro de senha MD5.
- c) registro de arquivo PDF.
- d) registro de página HTML.

Q60. O registro de conexão em SQL/CLI mantém:

- a) informações de controle de conexão de um BD em particular.
- b) informações sobre uma senha em texto puro.
- c) informações sobre imagens do sistema.
- d) apenas comentários do programa.

Q61. Procedimentos armazenados são:

- a) funções ou procedimentos armazenados localmente e executados pelo SGBD.
- b) comandos que só rodam no navegador.
- c) consultas digitadas apenas manualmente.
- d) chaves estrangeiras.

Q62. Uma vantagem dos procedimentos armazenados é:

- a) evitar duplicação quando o mesmo procedimento é usado por várias aplicações.
- b) tornar impossível o reuso por várias aplicações.
- c) impedir execução pelo servidor.
- d) eliminar completamente SQL.

Q63. Outra vantagem dos procedimentos armazenados é:

- a) reduzir custos de comunicação ao executar no servidor.
- b) aumentar obrigatoriamente o tráfego entre cliente e servidor.
- c) impedir qualquer chamada externa.
- d) remover o SGBD.

Q64. Uma desvantagem dos procedimentos armazenados é:

- a) problemas de portabilidade, pois cada SGBD pode ter sua própria sintaxe.
- b) impossibilidade de serem chamados por aplicações.
- c) ausência de execução no servidor.
- d) impossibilidade de uso com SQL.

Q65. O comando usado para chamar procedimento ou função armazenada é:

- a) **CALL**.

- b) `FETCH`.
- c) `DROP`.
- d) `CONNECT`.

Q66. SQL/PSM é:

- a) parte do padrão SQL para escrever módulos armazenados persistentes.
- b) uma API exclusiva de Java.
- c) uma técnica de ataque por strings maliciosas.
- d) uma forma de remover cursores.

Q67. SQL/PSM adiciona ao SQL:

- a) construções de programação como ramificações e loops.
- b) apenas comandos `DROP`.
- c) apenas comandos `HTML`.
- d) apenas criptografia MD5.

Q68. SQL Injection é:

- a) uma ameaça à segurança e integridade de sistemas que interagem com BD via SQL.
- b) uma técnica segura de criptografia de senha.
- c) uma forma normal de banco de dados.
- d) uma biblioteca de Java.

Q69. SQL Injection geralmente envolve:

- a) uso de strings maliciosas.
- b) uso obrigatório de chaves primárias.
- c) criação de índices.
- d) normalização até BCNF.

Q70. No exemplo da aula, a entrada `111111100 OR TRUE` em uma concatenação SQL é perigosa porque:

- a) pode transformar a condição em verdadeira para muitas tuplas.
- b) cria automaticamente uma chave estrangeira.
- c) impede qualquer exclusão.
- d) criptografa a senha corretamente.

Q71. Uma prática que ajuda a reduzir risco de SQL Injection é:

- a) usar comandos parametrizados, como `PreparedStatement`.
- b) concatenar diretamente toda entrada do usuário.

- c) aceitar qualquer string sem validação.
- d) usar `DELETE` sem `WHERE`.

Q72. Em dados criptografados no exemplo da aula, o *salt* serve para:

- a) compor o valor usado no hash da senha.
- b) substituir o nome do usuário.
- c) criar uma chave estrangeira.
- d) abrir conexão JDBC.

Q73. No exemplo de senha da aula, o valor armazenado no banco é:

- a) o hash da senha combinada com salt.
- b) a senha em texto puro.
- c) apenas o CPF do usuário.
- d) o comando SQL completo.

Q74. No login com senha criptografada, o servidor compara:

- a) o hash calculado da senha informada com o hash armazenado.
- b) a senha em texto puro com o CPF.
- c) a consulta SQL com o nome da tabela.
- d) a chave estrangeira com a chave primária.

2 Questões de Verdadeiro ou Falso

Q75. Julgue as afirmações:

- a) () A maioria das interações práticas com BD ocorre por programas de aplicação.
- b) () Interfaces web podem acessar bancos de dados.
- c) () A interface interativa é a única forma prática de acessar um BD.
- d) () Programação de BD é um assunto amplo e com várias técnicas.

Q76. Julgue as afirmações:

- a) () Java pode ser uma linguagem hospedeira.
- b) () SQL pode ser sublinguagem de dados.
- c) () SQL embutido mistura comandos SQL com linguagem hospedeira.
- d) () Linguagem hospedeira e SQL são sempre a mesma coisa.

Q77. Julgue as afirmações:

- a) () O descompasso de impedância envolve incompatibilidades entre modelos.
- b) () Resultados SQL com várias tuplas podem exigir cursores ou iteradores.

- c) () Tipos de dados podem contribuir para o descompasso de impedância.
- d) () O descompasso de impedância só ocorre em bancos NoSQL.

Q78. Julgue as afirmações:

- a) () SQL embutido pode usar `EXEC SQL`.
- b) () Variáveis compartilhadas podem ser prefixadas com dois pontos.
- c) () Um pré-processador pode separar SQL do código hospedeiro.
- d) () SQL embutido nunca permite consultas.

Q79. Julgue as afirmações:

- a) () `CONNECT` pode abrir conexão com servidor de BD.
- b) () `SET CONNECTION` pode mudar a conexão ativa.
- c) () `DISCONNECT` pode fechar uma conexão.
- d) () Apenas uma conexão pode existir em todo o programa.

Q80. Julgue as afirmações:

- a) () Cursor é útil para processar múltiplas tuplas.
- b) () `FETCH` move o cursor para a próxima tupla.
- c) () `CLOSE CURSOR` indica fim do processamento.
- d) () Cursor é usado apenas para apagar tabelas.

Q81. Julgue as afirmações:

- a) () SQL dinâmica permite montar comandos SQL em tempo de execução.
- b) () SQL dinâmica pode receber comandos digitados pelo usuário.
- c) () Consulta dinâmica pode ser complexa.
- d) () SQL dinâmica nunca usa strings.

Q82. Julgue as afirmações:

- a) () SQLJ é um padrão para embutir SQL em Java.
- b) () SQLJ é traduzido para Java usando JDBC.
- c) () SQLJ usa iteradores para múltiplas tuplas.
- d) () SQLJ é uma técnica exclusiva de criptografia.

Q83. Julgue as afirmações:

- a) () JDBC significa Java Database Connectivity.
- b) () JDBC usa drivers para acessar SGBDs.
- c) () `ResultSet` representa resultado de consulta.
- d) () `executeQuery` é usado principalmente para `INSERT`.

Q84. Julgue as afirmações:

- a) () Hibernate facilita o mapeamento objeto-relacional.
- b) () Hibernate pode gerar chamadas SQL.
- c) () Hibernate pode melhorar portabilidade entre SGBDs.
- d) () Hibernate elimina completamente a existência de tabelas.

Q85. Julgue as afirmações:

- a) () SQL/CLI é Call Level Interface.
- b) () SQL/CLI faz parte do padrão SQL.
- c) () SQL/CLI usa comandos SQL criados dinamicamente e passados como strings.
- d) () SQL/CLI não possui componentes de controle.

Q86. Julgue as afirmações:

- a) () Procedimentos armazenados são executados pelo SGBD.
- b) () Procedimentos armazenados podem reduzir duplicação.
- c) () Procedimentos armazenados podem reduzir custo de comunicação.
- d) () Procedimentos armazenados não possuem problemas de portabilidade.

Q87. Julgue as afirmações:

- a) () SQL/PSM permite escrever módulos armazenados persistentes.
- b) () SQL/PSM adiciona ramificações e loops ao SQL.
- c) () SQL/PSM aumenta o poder de SQL.
- d) () SQL/PSM é o mesmo que SQL Injection.

Q88. Julgue as afirmações:

- a) () SQL Injection ameaça segurança e integridade.
- b) () SQL Injection pode usar strings maliciosas.
- c) () Concatenar entrada do usuário diretamente em SQL pode ser perigoso.
- d) () PreparedStatement aumenta o risco de SQL Injection.

Q89. Julgue as afirmações:

- a) () Senhas podem ser armazenadas como hash com salt.
- b) () No login, pode-se comparar o hash calculado com o hash armazenado.
- c) () Armazenar senha em texto puro é sempre a melhor prática.
- d) () HTTPS aparece no fluxo de envio de dados no exemplo da aula.

3 Questões de Aplicação Objetiva

Q90. Considere o trecho:

```
String sql = "DELETE FROM EMPLOYEE WHERE ssn=" + ssn;
```

Qual é o principal problema desse código?

- a) Risco de SQL Injection por concatenação direta da entrada.
- b) Uso obrigatório de chave estrangeira.
- c) Ausência de `GROUP BY`.
- d) Uso de `SELECT` em vez de `DELETE`.

Q91. Considere a entrada:

```
111111100 OR TRUE
```

em um comando montado por concatenação. O efeito provável é:

- a) tornar a condição verdadeira para muitas tuplas.
- b) impedir execução do SQL para sempre.
- c) criar uma nova tabela chamada `TRUE`.
- d) transformar o comando em `SELECT`.

Q92. Qual alternativa representa uma forma mais segura de executar comandos com parâmetros em JDBC?

- a) Usar `PreparedStatement` com `?` e ligação de parâmetros.
- b) Concatenar diretamente qualquer entrada do usuário.
- c) Usar `DELETE` sem `WHERE`.
- d) Criar uma string SQL com `OR TRUE`.

Q93. Em JDBC, para uma consulta que retorna linhas, qual sequência faz mais sentido?

- a) criar conexão, criar comando, executar `executeQuery`, processar `ResultSet`.
- b) criar `ResultSet`, executar `DROP`, abrir conexão no final.
- c) fechar conexão, processar resultado, executar consulta.
- d) executar `DELETE`, usar `GROUP BY`, criar driver.

Q94. Em SQL embutido, qual comando é usado para avançar sobre tuplas de uma consulta com várias linhas?

- a) `FETCH`.
- b) `DROP`.
- c) `CREATE FUNCTION`.

d) `CALL`.

Q95. Em SQLJ, se o iterador lista tipos e nomes dos atributos, ele é:

- a) designado.
- b) posicional.
- c) dinâmico.
- d) criptográfico.

Q96. Em SQLJ, se o iterador lista somente os tipos dos atributos, ele é:

- a) posicional.
- b) designado.
- c) armazenado.
- d) injetado.

Q97. Qual técnica é mais diretamente associada ao mapeamento entre classes Java e tabelas relacionais?

- a) Hibernate.
- b) SQL Injection.
- c) `FETCH`.
- d) `SQLCODE`.

Q98. Qual alternativa descreve melhor um procedimento armazenado?

- a) Módulo armazenado e executado pelo SGBD.
- b) Consulta digitada apenas no terminal.
- c) Tabela virtual derivada de consulta.
- d) Senha criptografada com MD5.

Q99. Em um sistema de login com senha armazenada como hash com salt, o banco deve armazenar preferencialmente:

- a) o hash calculado a partir da senha e do salt.
- b) a senha original em texto puro.
- c) o comando SQL usado no login.
- d) o nome da tabela de usuários.

Q100. Considere o trecho JDBC:

```
Connection con = DriverManager.getConnection(url, user, pass);
```

O objeto `Connection` representa:

- a) uma conexão ativa com o banco de dados.

- b) uma linha retornada por consulta.
- c) uma tabela temporária.
- d) uma chave estrangeira.

Q101. Considere o trecho:

```
PreparedStatement ps = con.prepareStatement("SELECT * FROM Aluno WHERE matricula = ?");
```

O símbolo ? representa:

- a) um parâmetro a ser ligado antes da execução.
- b) um erro obrigatório de sintaxe.
- c) uma coluna chamada interrogação.
- d) um comando para apagar a tabela.

Q102. No trecho:

```
ps.setInt(1, matricula);
```

o número 1 indica:

- a) a posição do primeiro parâmetro ? no comando preparado.
- b) o número de linhas da tabela.
- c) o tipo da chave primária.
- d) o número de conexões abertas.

Q103. Considere:

```
ResultSet rs = ps.executeQuery();
```

Esse método é adequado principalmente para:

- a) comandos **SELECT** que retornam linhas.
- b) comandos **UPDATE** que não retornam linhas.
- c) fechar a conexão.
- d) criar uma classe Java.

Q104. Para percorrer as linhas retornadas em um **ResultSet**, usa-se normalmente:

- a) `while (rs.next())`.
- b) `while (con.close())`.
- c) `DROP RESULTSET`.
- d) `GROUP BY rs`.

Q105. Para executar um **UPDATE**, **INSERT** ou **DELETE** em **JDBC**, o método mais apropriado é:

- a) `executeUpdate()`.
- b) `executeQuery()` obrigatoriamente.
- c) `next()`.

d) `getString()`.

Q106. Considere o código:

```
String sql = "SELECT * FROM Usuario WHERE login='" + login + "' AND senha='" + sen
```

O principal risco é:

- a) SQL Injection por concatenação de entrada externa.
- b) violação automática da 1FN.
- c) ausência de `ORDER BY`.
- d) uso de transação serial.

Q107. Uma entrada como:

```
' OR '1'='1
```

em um login vulnerável pode:

- a) tornar a condição sempre verdadeira e burlar a autenticação.
- b) criar uma chave primária automaticamente.
- c) impedir qualquer consulta `SELECT`.
- d) normalizar a tabela de usuários.

Q108. Qual alternativa reduz melhor o risco de SQL Injection em JDBC?

- a) Usar `PreparedStatement` e parâmetros, evitando concatenação direta.
- b) Remover todos os comandos `WHERE`.
- c) Usar `Statement` com concatenação livre.
- d) Armazenar senha em texto puro.

Q109. Considere uma aplicação que precisa garantir que duas atualizações ocorram juntas. Em JDBC, uma abordagem adequada é:

- a) desativar `autoCommit`, executar as operações e chamar `commit()` ou `rollback()`.
- b) usar sempre `SELECT DISTINCT`.
- c) fechar a conexão antes das atualizações.
- d) executar cada atualização em bancos diferentes sem transação.

Q110. No JDBC, se uma exceção ocorre no meio de uma transação manual, a ação adequada para desfazer as alterações parciais é:

- a) chamar `rollback()`.
- b) chamar `getString()`.
- c) chamar `next()`.
- d) chamar `GROUP BY`.

Q111. Em SQL embutido, o uso de cursor é necessário principalmente quando:

- a) uma consulta retorna múltiplas tuplas e o programa precisa processá-las uma a uma.
- b) o comando é `DROP TABLE`.
- c) não existe linguagem hospedeira.
- d) a consulta retorna sempre uma constante.

Q112. Considere a sequência de cursor:

`DECLARE cursor, OPEN, FETCH, CLOSE`

O comando `FETCH` serve para:

- a) obter a próxima tupla do resultado.
- b) apagar o cursor fisicamente do disco.
- c) abrir uma nova conexão JDBC.
- d) executar `COMMIT` automaticamente.

Q113. Uma stored procedure é útil quando se deseja:

- a) armazenar lógica de processamento no SGBD para ser chamada por aplicações.
- b) eliminar todas as tabelas.
- c) substituir qualquer índice por imagem.
- d) impedir transações.

Q114. Em SQL/PSM, procedimentos e funções armazenadas permitem:

- a) usar estruturas procedurais próximas ao banco, como variáveis e controle de fluxo.
- b) apenas criar arquivos PDF.
- c) remover a necessidade de SQL.
- d) impedir chamadas por aplicações.

Q115. Hibernate é associado principalmente a:

- a) mapeamento objeto-relacional entre classes e tabelas.
- b) detecção de deadlock por grafo de espera.
- c) normalização para BCNF.
- d) ordenação por `ORDER BY`.

Q116. O problema conhecido como descompasso de impedância aparece porque:

- a) linguagens de programação e bancos relacionais organizam dados de formas diferentes.
- b) SQL e Java possuem exatamente o mesmo modelo de dados.
- c) todo banco relacional dispensa tipos.
- d) cursores eliminam objetos.

Q117. Para armazenar senhas com mais segurança, uma prática melhor que texto puro é:

- a) armazenar hash da senha com salt apropriado.
- b) armazenar a senha original sem proteção.
- c) colocar a senha no nome da tabela.
- d) usar `SELECT *` no login.

Q118. Se uma aplicação compara diretamente a senha digitada com uma senha em texto puro armazenada no banco, o problema principal é:

- a) exposição grave caso o banco seja vazado.
- b) excesso de normalização.
- c) uso obrigatório de cursor.
- d) conflito *write-write*.

Q119. Em uma consulta JDBC, chamar `rs.getString("nome")` normalmente significa:

- a) recuperar o valor da coluna `nome` da linha atual do `ResultSet`.
- b) criar uma coluna chamada `nome`.
- c) abrir uma conexão com senha `nome`.
- d) executar `DELETE` na tabela `nome`.

Gabarito

Q1. a	Q29. a
Q2. a	Q30. a
Q3. a	Q31. a
Q4. a	Q32. a
Q5. a	Q33. a
Q6. a	Q34. a
Q7. a	Q35. a
Q8. a	Q36. a
Q9. a	Q37. a
Q10. a	Q38. a
Q11. a	Q39. a
Q12. a	Q40. a
Q13. a	Q41. a
Q14. a	Q42. a
Q15. a	Q43. a
Q16. a	Q44. a
Q17. a	Q45. a
Q18. a	Q46. a
Q19. a	Q47. a
Q20. a	Q48. a
Q21. a	Q49. a
Q22. a	Q50. a
Q23. a	Q51. a
Q24. a	Q52. a
Q25. a	Q53. a
Q26. a	Q54. a
Q27. a	Q55. a
Q28. a	Q56. a
	Q57. a
	Q58. a

Q59. a	Q86. a
Q60. a	Q87. a
Q61. a	Q88. a
Q62. a	Q89. a
Q63. a	Q90. a
Q64. a	Q91. a
Q65. a	Q92. a
Q66. a	Q93. a
Q67. V, V, F, V	Q94. a
Q68. V, V, V, F	Q95. a
Q69. V, V, V, F	Q96. a
Q70. V, V, V, F	Q97. a
Q71. V, V, V, F	Q98. a
Q72. V, V, V, F	Q99. a
Q73. V, V, V, F	Q100. a
Q74. V, V, V, F	Q101. a
Q75. V, V, V, F	Q102. a
Q76. V, V, V, F	Q103. a
Q77. V, V, V, F	Q104. a
Q78. V, V, V, F	Q105. a
Q79. V, V, V, F	Q106. a
Q80. V, V, V, F	Q107. a
Q81. V, V, F, V	Q108. a
Q82. a	Q109. a
Q83. a	Q110. a
Q84. a	Q111. a
Q85. a	